DSA STUDY BLUEPRINT SERIES

16. Aggressive Cows Problem

Maximizing the Minimum Distance Between Stalled Cows

Section 02 • C++ Core Data Structures & Algorithms

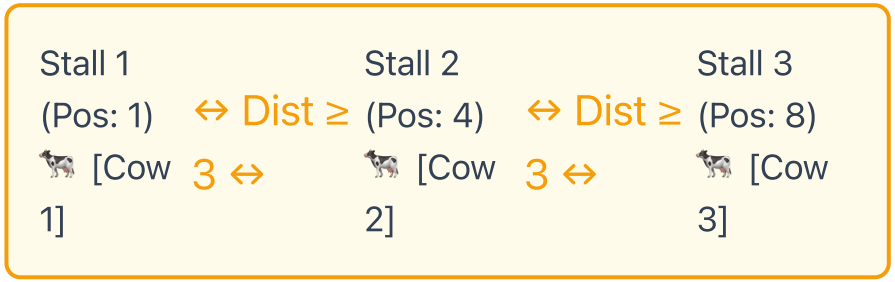
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Place C cows in stalls such that minimum distance between them is maximized.
- **Sorting:** Sort the stall coordinates first.
- Binary search distance between 1 and `stalls[last] - stalls[first]`.

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
bool canPlace(vector& stalls, int cows, int minDist) {
    int count = 1, lastPos = stalls[0];
    for (int i = 1; i < stalls.size(); i++) {
        if (stalls[i] - lastPos >= minDist) {
            count++; lastPos = stalls[i];
            if (count == cows) return true;
        }
    }
    return false;
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	O(N log(Max-Min) + N log N)
Space Complexity	O(1)

Key Practices & Gotchas

- **Important:** Sorting the stalls is a mandatory first step to enable monotonic distance checking.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run Binary Search on Answers range checks (e.g. allocating book pages):

Iteration	Check Bounds [Low, High]	Mid Candidate Value	Feasibility Check: `isValid(mid)`	Search Window Adjustment
1	[90, 203]	mid = 146	True (Students needed ≤ K)	Shrink search: high = mid - 1 = 145
2	[90, 145]	mid = 117	False (Students needed > K)	Expand search: low = mid + 1 = 118

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Minimizing the Maximum:** Used when the answer search space is monotonic (if X is feasible, all values > X are also feasible). Search binary-wise for bounds limits.
- **Feasibility Helpers:** Write custom O(N) linear scan checks to evaluate candidate value viability.
- **Related LeetCode Problems:** #410 Split Array Largest Sum, #1011 Capacity To Ship Packages Within D Days, #875 Koko Eating Bananas.